

STUDENT CASE STUDY 3: DEVELOPING API FOR INTEGRATING PAYROLL AND HUMAN DATA

Course: CMU-CS 445 – *System Integration Practices*

Semester: 2

Academic: 2024–2025

Instructor: MSc. Nguyen Dang Quang Huy

1. Objective

The purpose of this case study is to develop a functional backend API that connects two enterprise databases, HUMAN (SQL Server) and PAYROLL (MySQL), enabling synchronization and real-time data exchange between HR and payroll systems.

Students are required to:

- Build RESTful APIs according to specifications from Case Study 2.
- Implement data synchronization logic across heterogeneous databases.
- Handle validation, transaction rollback, and error reporting.
- Conduct unit and integration testing to verify system accuracy and reliability.

2. Scenario Background

After completing the integration model in Case Study 2, Company X requires a backend implementation to operationalize the designed APIs.

The backend must securely connect the HR and Payroll databases, synchronize updates for employee and salary data, and provide reliable API endpoints for future dashboard integration.

The system must ensure:

- Consistent and secure data flow between SQL Server and MySQL.
- Validation for create, update, and delete operations.
- Proper rollback handling in case of synchronization or connection failure.
- Comprehensive testing and logging for traceability and maintenance.

3. Tasks and Requirements

3.1 Backend API Development

Students must implement a backend using either C# (.NET 8 Web API) or Python (Flask or FastAPI).

The system should connect to both databases:

- SQL Server for HUMAN (managing Employees, Departments, and Job Titles).
- MySQL for PAYROLL (managing Payroll, Attendance, and Salary Records).

The following API endpoints are required:

- GET /employees: Retrieve employee list from HUMAN.
- GET /payroll: Retrieve payroll data from PAYROLL.
- POST /add-employee: Add a new employee to HUMAN and synchronize with PAYROLL.
- PUT /update-employee: Update employee data in both databases.
- DELETE /delete-employee: Delete an employee if no dependent payroll or dividend data exists.
- GET /attendance: Retrieve attendance data from PAYROLL.
- GET /reports: Generate HR, payroll, and shareholder reports.
- POST /alerts: Send alerts for HR, leave, or payroll discrepancies.

3.2 Data Synchronization and Processing

The API must ensure atomic, consistent, and reversible transactions across the two databases.

Test scenarios include:

- Adding a new employee with valid data: API must insert data into both SQL Server and MySQL.
- Missing required fields such as EmployeeID, Name, Department, JobTitle, or BaseSalary: API returns 400 Bad Request.
- Duplicate EmployeeID in either database: API returns 409 Conflict.
- Connection error with SQL Server or MySQL: API performs rollback to prevent inconsistent data.
- Updating salary or job title: changes must synchronize across both systems.
- Deleting employee with linked payroll or dividend data: API must block the operation and return an appropriate warning message.

Transaction handling requirements:

- Use database transactions for all multi-database operations.
- If an error occurs, rollback must restore previous data state.
- Maintain system logs to record all API activity and error events.

3.3 Validation and Error Handling

Implement strict data validation before processing requests.

- Validate data type, completeness, and business rules.
- Return clear and standardized HTTP responses:
 - 200 OK for success.
 - 400 Bad Request for invalid input.
 - 404 Not Found for missing records.
 - 409 Conflict for duplication or constraint violations.
 - 500 Internal Server Error for database or transaction issues.

3.4 Unit Testing and API Testing

Students must write both unit tests and integration tests to ensure reliability.

Unit testing:

- Use xUnit or NUnit (for C#) or pytest/unittest (for Python).
- Test core functions including database connection, insert, update, and delete logic.

API testing:

- Use Postman, Swagger, or Insomnia.
- Verify request-response consistency, error handling, and synchronization results.
- Confirm that all data changes are reflected correctly in both databases.

Sample test cases include:

- Adding a valid employee and checking if data appears in both databases.
- Attempting to add a duplicate employee ID and expecting a 409 Conflict response.
- Updating salary and verifying synchronization.
- Deleting employees with and without payroll dependencies to test constraint logic.

3.5 Documentation and Reporting

Each group must submit:

- API documentation describing all endpoints, input/output parameters, and response codes.
- Technical report explaining system architecture, synchronization process, and testing methodology.
- Log files or screenshots showing successful API operations and rollback scenarios.

4. Expected Deliverables

4.1 Document Deliverables

Document Name	Action (New / Update)	Detailed Description / Student Instructions
8. API Design & Implementation Document	New	Create APIs that connect HUMAN (SQL Server) and PAYROLL (MySQL). For each API (GET, POST, PUT, DELETE), write its route, purpose, required parameters, and example input/output. Example: POST /add-employee – adds an employee to both databases; returns success message if synchronization is successful. Document all error messages (400, 409, 500) and rollback logic when a database connection fails.
9. Test Plan Document	New	Describe how you will test the APIs. Include testing goals (e.g., data synchronization), testing environment (tools like Postman, SQL Server, MySQL Workbench), and testing types (unit test, integration test). Plan test frequency and expected outcomes.
10. Test Case Document	New	List detailed test cases in a table with columns: <i>Test ID</i> , <i>API name</i> , <i>Input data</i> , <i>Expected result</i> , <i>Actual result</i> , <i>Status</i> . Include examples of valid and invalid inputs (e.g., missing required fields, duplicate EmployeeID). Attach screenshots of testing results if possible.

11. Updated SRS Document (Version 2)	Update from Case 2	Add new technical sections showing database connection setup, API architecture, and synchronization flow. Include diagrams of how the backend exchanges data between HUMAN and PAYROLL.
12. Updated Data Mapping Document	Update from Case 2	Add the exact fields used in your API code. For example, document how the “Salary” table in PAYROLL maps to “EmployeeSalary” in HUMAN. Mention primary/foreign keys used for synchronization.

4.2 Software Functional Deliverables

Software Functions to Develop	Description / Implementation Guidance
1. Employee Management (API Layer)	Implement CRUD operations with real databases: • GET /employees – list all employees from HUMAN. • POST /add-employee – add new employee to both HUMAN and PAYROLL. • PUT /update-employee – update employee info across both systems. • DELETE /delete-employee – remove employee (with constraint validation).
2. Payroll Management (API Layer)	Implement payroll endpoints: • GET /payroll – get salary data from PAYROLL. • PUT /update-salary – modify employee salary and synchronize to HR system.
3. Attendance Management (API Layer)	Implement attendance retrieval: • GET /attendance – get data from PAYROLL (working days, leave, absences).
4. Reporting & Alerts (API Layer)	Create initial data endpoints: • GET /reports – return consolidated HR–Payroll reports. • POST /alerts – trigger or record system alerts.

5. Data Synchronization Logic	Ensure API consistency: • If one database fails, rollback transaction. • Check for duplicate employee IDs, missing required fields, and log errors.
--------------------------------------	---

5. Evaluation Criteria

- API functionality and completeness.
- Accuracy and consistency of data synchronization.
- Error handling and rollback mechanism.
- Quality and coverage of testing activities.
- Code readability and documentation quality.